

Lab 6: Is extreme rainfall at Hobby changing?

2026-10-02

In this lab, you will build your own nonstationary model of extreme daily rainfall at Houston Hobby Airport and decide whether the evidence supports a published claim that it is rising. Everyone uses the same record, and each of you makes different choices along the way. On Wednesday, October 7, you will present your results in class and defend those choices, as practice for the oral exam.

Objectives

By the end of this lab, you will be able to:

1. Build a nonstationary extreme value model, choosing annual maxima or peaks over threshold, a covariate, and the form of the trend.
2. Report a change in the 100-year rainfall with an interval, and say what the interval leaves out.
3. Defend each choice behind your result.

Setup

First, load the packages.

```
using Pkg
here = dirname(@__FILE__)
Pkg.activate(here)
Pkg.instantiate()

using CEVE543Utils
using CairoMakie, CSV, DataFrames, Dates, Distributions, LaTeXStrings
using Printf, Random, Statistics, Turing
using FlexiChains: summarystats

rng = MersenneTwister(543)
set_theme!(theme_minimal(); fontsize=14)
colors = Makie.wong_colors()
```

Rainfall data

You will analyze daily precipitation at Houston Hobby Airport, station [USW00012918](#) in NOAA's Global Historical Climatology Network, which is the record from Monday's lectures. The file `hobby-daily.csv` has one row for each day with a valid reading, in millimeters.

?@fig-daily plots every wet day.

Not every year is complete. ?@fig-days-per-year counts the days with a valid reading in each year.

Covariates

The file `covariates.csv` has three candidate covariates, one row per year from 1880 to 2025:

Column	What it is	Source
<code>log_CO2</code>	Natural log of annual mean atmospheric CO ₂ , in parts per million	Before 1959, the Law Dome ice core record [1], [2]; from 1959, the Mauna Loa annual means from NOAA's Global Monitoring Laboratory
<code>gmst</code>	Global mean surface temperature anomaly, in °C relative to 1951-1980	NASA's GISTEMP version 4, January to December mean
<code>nino34</code>	Sea surface temperature anomaly in the Niño 3.4 region of the tropical Pacific, in °C relative to 1981-2010	HadISST, from NOAA's Physical Sciences Laboratory, mean of the 12 calendar months

CO₂ and global temperature both rise over the record, so they are candidates for a long-term trend. Niño 3.4 measures the El Niño-Southern Oscillation, which shifts from year to year with no long-term trend, so it is a candidate for explaining which years are wet.

Your Challenge

C. Fagnant, A. Gori, A. Sebastian, P. B. Bedient, and K. B. Ensor [3] analyzed more than 600 rain gauges in southeast Texas and reported that the 100-year daily rainfall at Hobby rose by 1.93 mm per year (95% interval 1.50 to 2.35), with several methodological caveats. In this lab, you will analyze a similar dataset and fit your own model. You will make key choices, including whether to fit a Generalized Extreme Value (GEV) or Generalized Pareto Distribution (GPD) model, what covariates if any to use (and their functional form), and any priors to include.

Prepare a 5-10 minute slide presentation for class on Wednesday, October 7, that leads with your results and then explains your modeling choices. **Key questions to answer are:**

1. What is your best estimate for the 10- and 100-year return levels in 2026?
2. How uncertain are these estimates?
3. How have these return levels changed over the past 40 years?
4. How will these return levels change over the next 40 years, and what assumptions do you make in projecting these?

Explain why you made each choice, as well as what it was. **Key decisions to explain include:**

1. Did you fit a GEV to annual maxima or GPD to peaks over threshold?
 1. If GEV:
 1. How did you handle years with missing data?
 2. If GPD:
 1. How did you decluster?
 2. How did you select the threshold?

2. How did you decide whether to fit a nonstationary or stationary model?
 1. If nonstationary:
 1. How did you choose the covariates?
 2. How did you choose the functional form?
 2. If stationary:
 1. What gave you confidence in this choice?
3. How do you characterize uncertainty?
 1. If Bayesian: how did you choose priors?
 2. If bootstrap: how did you choose how to resample?

Examples

This is a relatively open-ended lab, particularly compared to your other labs. Two worked examples on synthetic (simulated) data give you all the code you need. You may **delete this section from your final notebook**.

GEV Example

The first example is a GEV with a quadratic trend in the location and the log scale. A quadratic trend is usually a bad idea, but it shows how to use the software.

$$y_t \sim \text{GEV}(\alpha_0 + \alpha_1 x_t + \alpha_2 x_t^2, \exp(\beta_0 + \beta_1 x_t + \beta_2 x_t^2), \xi).$$

For simplicity, the covariate is the year centered on 1985, $x_t = t - 1985$. The synthetic dataset has one annual maximum for each year from 1946 to 2025. In 1985 its location rises by 0.5 mm per year and its scale grows by 0.4% per year, and both rates speed up slowly over time.

The `gevfitbayes` function from the `CEVE543Utils` package fits a GEV whose location and log scale are each a linear combination of the covariates you pass. Passing x_t and x_t^2 as two covariates gives the quadratic above:

where

- `locationcov` is a list of covariates for the location parameter (`[x, x.^2]` is a list holding two covariates)
- `logscalecov` is a list of covariates for the log-scale parameter
- `rng` specifies the random number generator
- `progress` specifies whether to show a progress bar.

By default, priors in `gevfitbayes` are flat on every coefficient and uniform between -0.5 and 0.5 on ξ . The posterior is sampled with an implementation of DE-MCzs, the sampler in the RMC-BestFit software that C. H. Smith, B. Skahill, and D. A. Margo [4] use, although you can change this by passing in `sampler=NUTS()` to use the NUTS sampler.

The `summarystats` table is for checking the chains. Check the `rhat` column, which should be close to 1, and the `ess_bulk` column, which should be in the hundreds or more, as in lab 5. DE-MCzs takes no gradient steps, so it has no divergences to count. The coefficients in that table are on the standardized scale the fit works on internally.

To read the coefficients in the units of the covariate, use `params`. It gives one row per posterior draw for each parameter. The first column is the intercept, which is the value at the means of the covariates rather than at $x_t = 0$. The second column is α_1 , the slope per year in 1985, and the third is α_2 .

The projection to 2066 assumes the quadratic keeps going for another 40 years, past the end of the data. With log CO2 or GMST as the covariate, you would need a value for that covariate in 2066, which is an assumption about the future that you have to state and defend.

`?@fig-syn-trend` plots the fitted 2- and 100-year levels in each year of the record next to the true levels the synthetic data came from.

If you don't want to use a Bayesian approach, you can instead fit by maximum likelihood and use bootstrapping or another approach to characterize your uncertainty. `gevfit` fits the same model by maximum likelihood, and `getdistribution` gives the fitted GEV for each year. A parametric bootstrap draws a new record from those distributions, refits it, and records its return levels, many times over. The spread of the refitted levels is the uncertainty.

Other functional forms

The functional form is which parameters change with the covariate, and how. Every fitting function also takes a table and the names of its columns. For example, `gevfitbayes(annmax, :y; locationcovid=[:log_CO2], logscalecovid=[:log_CO2])` fits the column `y` of `annmax` with the location and the log scale each a line in `log_CO2`. With this table form, each functional form is a choice of covariate columns:

- A trend in the location alone passes `locationcovid` and leaves out `logscalecovid`.
- A trend in the scale alone does the opposite.
- A curved trend adds a column holding the covariate squared to your table, such as `annmax.log_CO2_sq = annmax.log_CO2 .^ 2`, and names it in `locationcovid`. A quadratic will find some curvature in almost any record, including one drawn from a straight line. Outside the data it grows without limit, so its 2066 level rests on curvature the record barely constrains.
- An abrupt change at a year you choose adds a column that is 0 before that year and 1 after it. Say why that year, and whether you chose it before or after looking at the data.

Prior information

Two keywords change the priors. `trend_priors` puts a prior on the slopes, one for each parameter, and any you leave out stay flat. The slope priors are on the standardized scale of the `summarystats` table, so each is a change per standard deviation of the covariate. A `location` slope is in millimeters, so `trend_priors=(location=Normal(0, 5),)` says the location probably moves less than about 10 mm per standard deviation. A `logscale` slope is a proportional change in the scale, so `logscale=Normal(0, 0.1)` says the scale probably changes by less than about 20% per standard deviation. `quantile_priors` states what you believe about return levels, as in lab 4. Each belief names an observation, a return period, and a distribution for that return level in the year of the observation, and you can give as many as you like.

GPD Example

The second example is a GPD with a trend in the log scale, using the same centered year as the covariate. The synthetic storm record has 12 storms a year above a threshold of 25.4 mm, and the scale of each storm's excess grows by 0.4% per year.

With 960 storms in place of 80 annual maxima, the intervals are narrower.

Your Hobby analysis

Now use the code in the examples to answer the key questions above. If you fit a GEV, this cell makes a table of annual maxima joined to the covariates:

1. Return levels in 2026

The estimated 10-year return level is about 195 mm, and the 100-year return level is about 342 mm. 2. Uncertainty The 95% credible intervals are approximately 173–233 mm and 265–522 mm, respectively. The 100-year estimate is more uncertain because it describes a rarer event. 3. Changes over the past 40 years My stationary model estimates no change. The tests found no significant trend, but this does not prove that no change occurred. 4. Changes over the next 40 years The model projects no change if the climate and physical processes driving extreme rainfall remain similar. 5. Why GPD? I used GPD because it includes multiple extreme events per year, rather than only the annual maximum. I kept years with at least 350 valid days. 6. Threshold selection I chose 57.2 mm, the 99th percentile. The mean excess plot was roughly linear near this threshold. Parameter stability plots provided additional threshold diagnostics. 7. Declustering I grouped consecutive exceedances separated by 72 hours or less and kept the largest value in each group. This helps avoid counting the same storm more than once. 8. Why stationary? The trend tests found no significant trends, so I considered a stationary model. These tests do not prove stationarity. 9. Priors and uncertainty I used the Lab05 priors: $N(0, 3^2)$ for log scale and $N(0, 0.25^2)$ for shape. I calculated return levels from the posterior draws and used their 2.5th and 97.5th percentiles for the credible intervals. These intervals treat the event rate as fixed. ### Setup and retained data

```
# Load the record because the earlier loading cell is commented out.
daily = CSV.read(joinpath(here, "hobby-daily.csv"), DataFrame)

@assert allunique(daily.date) "Duplicate dates must be resolved."

# Keep valid daily rainfall observations through 2025.
daily = filter(
  r -> !ismissing(r.date) &&
    !ismissing(r.prcp_mm) &&
    isfinite(r.prcp_mm) &&
    r.prcp_mm >= 0 &&
    year(r.date) <= 2025,
  daily
)

daily.year = year.(daily.date)

# Preserve the full valid record for checking rainfall between peaks.
daily_check = sort(copy(daily), :date)

# Retain years with at least 350 valid observations.
coverage = combine(
  groupby(daily_check, :year),
```

```

    nrow => :n_days
  )

  retained_years = sort(
    coverage.year[coverage.n_days .>= 350]
  )

  @assert !isempty(retained_years) "No eligible years were found."

  daily = filter(r -> r.year in retained_years, daily_check)

  # Calculate the threshold using all observed days, including zeros.
  rain = Float64.(daily.prcp_mm)
  u99 = quantile(rain, 0.99)

  rainy_counts = [
    count(r -> r.year == yr && r.prcp_mm > 0, eachrow(daily))
    for yr in retained_years
  ]

  @printf(
    "Retained years: %d\nAverage observed rainy days per year: %.1f\n99th-percentile\n"
    "threshold: %.2f mm\n",
    length(retained_years),
    mean(rainy_counts),
    u99
  )

```

```

Retained years: 80
Average observed rainy days per year: 101.8
99th-percentile threshold: 57.20 mm

```

Mean excess plot

```

thresholds = collect(0.0:1.0:180.0)
min_exceedances = 20

mean_excess = [
  begin
    excess = rain[rain .> u] .- u
    length(excess) >= min_exceedances ? mean(excess) : NaN
  end
  for u in thresholds
  ]

```

```

fig = Figure(size=(1000, 280), backgroundcolor=:white)

ax = Axis(
    fig[1, 1];
    xlabel="Threshold u (mm)",
    ylabel="Mean excess (mm)",
    xticks=0:30:180,
    yticks=0:10:60,
    xgridvisible=false,
    ygridcolor=(:gray, 0.15)
)

lines!(
    ax, thresholds, mean_excess;
    color="#168C91",
    linewidth=2.5
)

probs = [0.95, 0.96, 0.97, 0.98, 0.99]
threshold_quantiles = quantile(rain, probs)

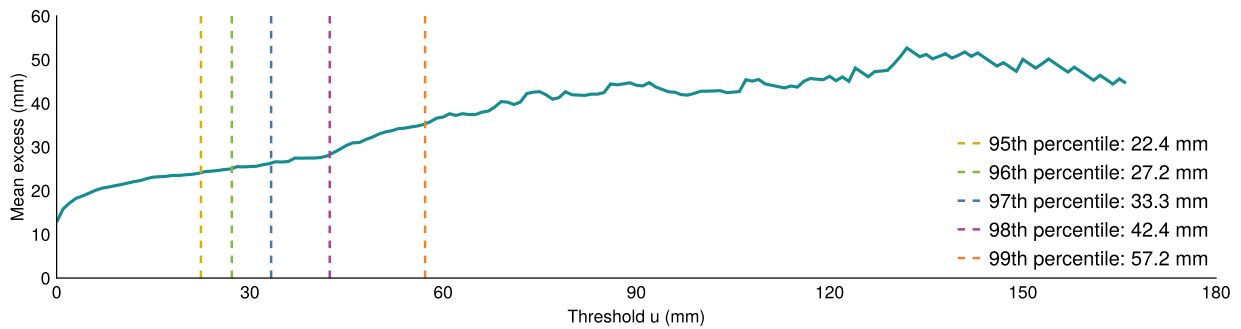
line_colors = [
    "#D4AE19",
    "#84B547",
    "#4477AA",
    "#AA4499",
    "#E68535"
]

for (p, u, color) in zip(probs, threshold_quantiles, line_colors)
    vlines!(
        ax, [u];
        color=color,
        linestyle=:dash,
        linewidth=2,
        label="$(round{Int, 100p})th percentile: $(round(u; digits=1)) mm"
    )
end

xlims!(ax, 0, 180)
ylims!(ax, 0, 60)
axislegend(ax; position=:rb, labelsz=16)
hidespines!(ax, :t, :r)

fig

```



Declustering and gap comparison

```
# Merge adjacent exceedances separated by at most the selected gap.
# Retain the largest daily rainfall value in each cluster.
```

```
function decluster_gap(exceedances, gap_hours)
  ordered = sort(exceedances, :date)
  isempty(ordered) && return copy(ordered)

  selected = Int[]
  best = 1

  for i in 2:nrow(ordered)
    separation_hours = 24 * Dates.value(
      ordered.date[i] - ordered.date[i - 1]
    )

    if separation_hours > gap_hours
      push!(selected, best)
      best = i
    elseif ordered.prcp_mm[i] > ordered.prcp_mm[best]
      best = i
    end
  end

  push!(selected, best)
  return ordered[selected, :]
end
```

```
exceedances99 = filter(r -> r.prcp_mm > u99, daily)
peaks72 = decluster_gap(exceedances99, 72)
peaks72.excess_mm = peaks72.prcp_mm .- u99
```

```
min_gap_hours = [24, 72, 168, 344]
palette = ["black", "black", "black", "black"]
```

```
# Use annual event counts for the gap-comparison figures.
```

```

# Exposure-adjusted rates are calculated separately below.
annual_lambda = DataFrame(
    percentile=Int[],
    threshold_mm=Float64[],
    gap_hours=Int[],
    year=Int[],
    lambda_per_year=Float64[]
)

for gap_h in min_gap_hours
    gap_peaks = decluster_gap(exceedances99, gap_h)

    for yr in retained_years
        n_events_year = count(==(yr), gap_peaks.year)

        push!(annual_lambda, (
            99,
            u99,
            gap_h,
            yr,
            Float64(n_events_year)
        ))
    end
end

@printf("Retained peaks with a 72-hour gap: %d\n", nrow(peaks72))

results99 = filter(r -> r.percentile == 99, annual_lambda)
ymax = 1.1 * maximum(results99.lambda_per_year) + 0.2

fig = Figure(size=(1600, 380), backgroundcolor=:white)

for (k, gap_h) in enumerate(min_gap_hours)
    results = sort(
        filter(r -> r.gap_hours == gap_h, results99),
        :year
    )

    mean_lambda = mean(results.lambda_per_year)

    local ax = Axis(
        fig[1, k];
        title="Gap: $gap_h h",
        xlabel="Year",
        ylabel=k == 1 ? "Events/year" : "",
        xgridvisible=false,

```

```

        ygridcolor=(:gray, 0.15),
        titlesize=16
    )

    scatterlines!(
        ax, results.year, results.lambda_per_year;
        color=palette[k],
        linewidth=1.8,
        markersize=4
    )

    hlines!(
        ax, [mean_lambda];
        color=:gray40,
        linestyle=:dash,
        linewidth=1.5
    )

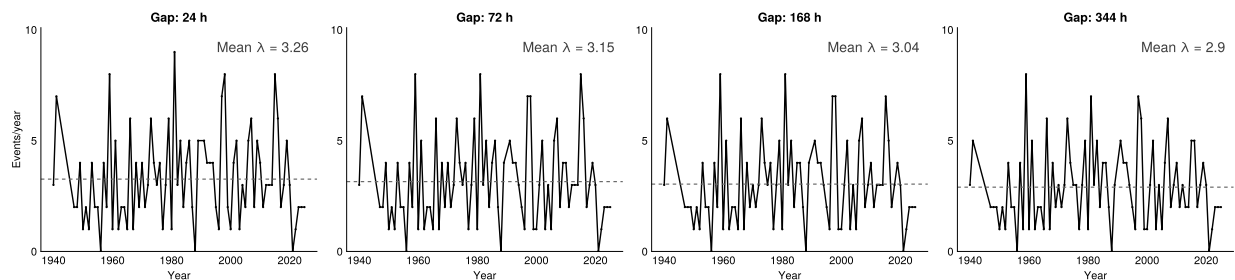
    text!(
        ax, 0.97, 0.95;
        text="Mean  $\lambda$  = $(round(mean_lambda; digits=2))",
        space=:relative,
        align=(:right, :top),
        fontsize=18,
        color=:gray25
    )

    ylims!(ax, 0, ymax)
    hidespines!(ax, :t, :r)
end

fig

```

Retained peaks with a 72-hour gap: 252



Retained peaks

```
fig = Figure(size=(1100, 400), backgroundColor=:white)

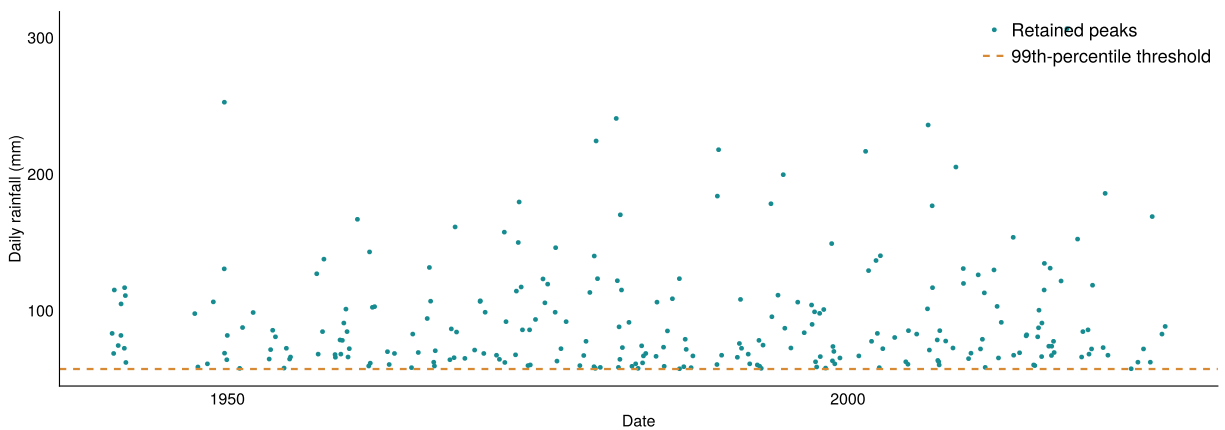
ax = Axis(
    fig[1, 1];
    xlabel="Date",
    ylabel="Daily rainfall (mm)",
    xgridvisible=false,
    ygridcolor=(:gray, 0.15)
)

scatter!(
    ax, peaks72.date, peaks72.prcp_mm;
    color="#168C91",
    markersize=6,
    label="Retained peaks"
)

hlines!(
    ax, [u99];
    color="#D88934",
    linestyle=:dash,
    linewidth=2,
    label="99th-percentile threshold"
)

axislegend(ax; position=:rt, labelsizes=16)
hidespines!(ax, :t, :r)

fig
```



Rainfall between retained peaks

```
between_minima = DataFrame(  
  date1=Date[],  
  date2=Date[],  
  min_rain_mm=Union{Missing, Float64}[],  
  n_missing_days=Int[]  
)  
  
rain_by_date = Dict(  
  r.date => r.prcp_mm for r in eachrow(daily_check)  
)  
  
for i in 2:nrow(peaks72)  
  d1 = peaks72.date[i - 1]  
  d2 = peaks72.date[i]  
  
  between_dates = collect(  
    (d1 + Day(1)):Day(1):(d2 - Day(1))  
  )  
  
  observed = Float64[]  
  n_missing = 0  
  
  for d in between_dates  
    value = get(rain_by_date, d, missing)  
  
    if ismissing(value)  
      n_missing += 1  
    else  
      push!(observed, Float64(value))  
    end  
  end  
  
  minimum_rain = isempty(observed) ? missing : minimum(observed)  
  
  push!(between_minima, (  
    d1, d2, minimum_rain, n_missing  
  ))  
end  
  
has_minimum = !ismissing.(between_minima.min_rain_mm)  
complete = has_minimum .& (between_minima.n_missing_days .== 0)  
incomplete = has_minimum .& (between_minima.n_missing_days .> 0)  
  
# Report how many intervals contain a recorded zero-rainfall day.
```

```

n_zero_intervals = count(
    v -> !ismissing(v) && v == 0,
    between_minima.min_rain_mm
)

@printf(
    "Intervals with a recorded zero-rainfall day: %d of %d\nIntervals containing missing
days: %d\n",
    n_zero_intervals,
    nrow(between_minima),
    count(>0), between_minima.n_missing_days)
)

fig_min = Figure(size=(1100, 400), backgroundcolor=:white)

ax_min = Axis(
    fig_min[1, 1];
    xlabel="Date of later retained peak",
    ylabel="Minimum rainfall between peaks (mm)",
    xgridvisible=false,
    ygridcolor=(:gray, 0.15)
)

if any(complete)
    scatter!(
        ax_min,
        between_minima.date2[complete],
        Float64.(between_minima.min_rain_mm[complete]);
        color="#168C91",
        markersize=7,
        label="Complete interval"
    )
end

if any(incomplete)
    scatter!(
        ax_min,
        between_minima.date2[incomplete],
        Float64.(between_minima.min_rain_mm[incomplete]);
        color="#D88934",
        marker=:utriangle,
        markersize=8,
        label="Interval with missing days"
    )
end

```

```

hlines!(ax_min, [0.0]; color=:gray40, linestyle=:dash)

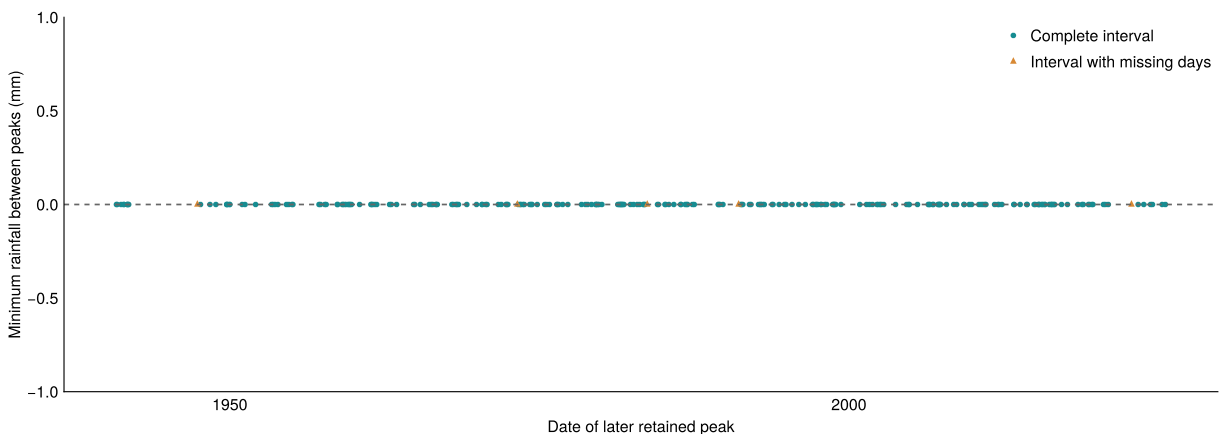
# Keep zero visible while allowing positive minima to be displayed.
observed_minima = Float64.(collect(skipmissing(between_minima.min_rain_mm)))
upper_limit = isempty(observed_minima) ? 1.0 :
    max(1.0, 1.1 * maximum(observed_minima))
ylims!(ax_min, -1, upper_limit)

axislegend(ax_min; position=:rt, labelsz=14)
hidespines!(ax_min, :t, :r)

fig_min

```

Intervals with a recorded zero-rainfall day: 251 of 251
Intervals containing missing days: 5



Trend assessment

```

annual_tests = combine(
    groupby(daily, :year),
    :prcp_mm => maximum => :max_mm,
    :date => (d -> length(unique(d))) => :n_days
)

event_counts = combine(
    groupby(peaks72, :year),
    nrow => :n_events
)

annual_tests = leftjoin(annual_tests, event_counts; on=:year)
annual_tests.n_events = Int.(coalesce.(annual_tests.n_events, 0))
sort!(annual_tests, :year)

```

```

annual_tests.exposure_years = [
    r.n_days / Dates.daysinyear(Date(r.year, 1, 1))
    for r in eachrow(annual_tests)
]

annual_tests.lambda = (
    annual_tests.n_events ./ annual_tests.exposure_years
)

# Mann-Kendall test with tie and continuity corrections.
function mk_test(response)
    y = Float64.(response)
    n = length(y)
    n >= 3 || error("At least three observations are required.")

    S = 0
    for i in 1:(n - 1)
        for j in (i + 1):n
            S += sign(y[j] - y[i])
        end
    end

    counts = Dict{Float64, Int}()
    for value in y
        counts[value] = get(counts, value, 0) + 1
    end

    tie_correction = sum(
        t * (t - 1) * (2t + 5)
        for t in values(counts)
    )

    var_S = (
        n * (n - 1) * (2n + 5) - tie_correction
    ) / 18

    z = if var_S == 0 || S == 0
        0.0
    else
        (S - sign(S)) / sqrt(var_S)
    end

    return (
        tau=S / (n * (n - 1) / 2),
        p_value=2 * cdf(Normal(), abs(z))
    )
end

```

```

    )
end

# Ordinary linear regression slope test.
function slope_test(years, response)
  x = Float64.(years)
  y = Float64.(response)
  n = length(y)

  n > 2 || error("At least three observations are required.")
  length(x) == n || error("Input lengths must match.")

  xc = x .- mean(x)
  sxx = sum(abs2, xc)
  sxx > 0 || error("Years must vary.")

  slope = sum(xc .* (y .- mean(y))) / sxx
  fitted = mean(y) .+ slope .* xc
  residuals = y .- fitted

  slope_se = sqrt(
    sum(abs2, residuals) / (n - 2) / sxx
  )

  t_stat = if slope_se > 0
    slope / slope_se
  elseif slope == 0
    0.0
  else
    sign(slope) * Inf
  end

  critical_t = quantile(TDist(n - 2), 0.975)

  return (
    slope=slope,
    lower95=slope - critical_t * slope_se,
    upper95=slope + critical_t * slope_se,
    p_value=2 * cdf(TDist(n - 2), abs(t_stat))
  )
end

trend_results = DataFrame(
  series=String[],
  mk_tau=Float64[],
  mk_p=Float64[],

```

```

    slope=Float64[],
    slope_lower95=Float64[],
    slope_upper95=Float64[],
    slope_p=Float64[]
  )

  for (label, response) in (
    ("Annual maximum rainfall", annual_tests.max_mm),
    ("Annual event rate", annual_tests.lambda)
  )
    mk = mk_test(response)
    reg = slope_test(annual_tests.year, response)

    push!(trend_results, (
      label,
      mk.tau,
      mk.p_value,
      reg.slope,
      reg.lower95,
      reg.upper95,
      reg.p_value
    ))
  end

  trend_results

```

```

2×7 DataFrame
 Row | series              mk_tau  mk_p    slope  slope_lower95 ...
   | String              Float64  Float64  Float64  Float64         ...
-----|-----
  1 | Annual maximum rainfall  0.101266  0.185044  0.387615   -0.115429 ...
  2 | Annual event rate       0.0268987  0.723243  0.00223756 -0.0163367 ...
                                     2 columns omitted

```

Threshold stability

```

u_base = quantile(rain, 0.95)
u_candidate = u99

# Use a common set of peaks declustered at a lower base threshold.
base_exceedances = sort(
  filter(r -> r.prcp_mm > u_base, daily),
  :date
)

```

```

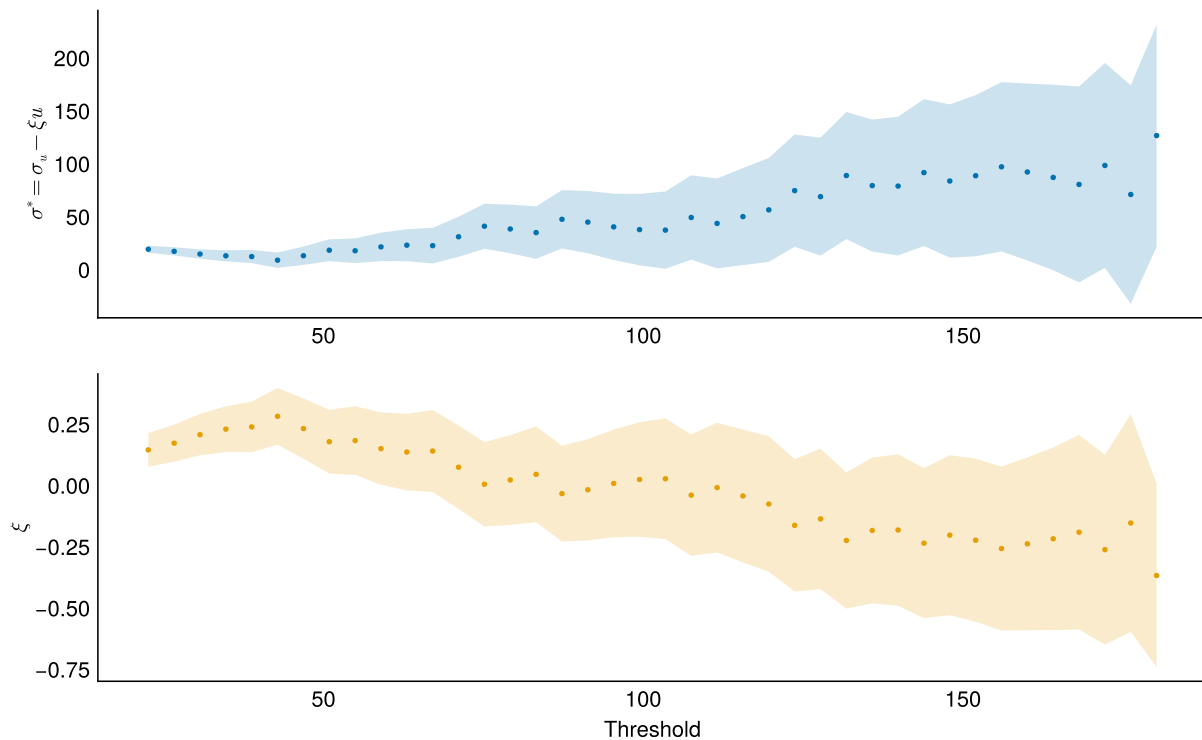
base_peaks = decluster_gap(base_exceedances, 72)
peak_values = Float64.(base_peaks.prcp_mm)

@printf(
    "Base threshold: %.1f mm\nCandidate threshold: %.1f mm\nEvents above candidate:
%d\n",
    u_base,
    u_candidate,
    count(v -> v > u_candidate, peak_values)
)

# Plot modified scale and shape with approximate confidence intervals.
stability_plot(
    peak_values;
    lo_quantile=0.0,
    hi_quantile=0.99
)

```

Base threshold: 22.4 mm
 Candidate threshold: 57.2 mm
 Events above candidate: 251



Bayesian stationary GPD

```
excess_final = Float64.(peaks72.prcp_mm .- u99)

@assert all(excess_final .> 0)

# Estimate the constant event rate using observed exposure.
exposure_final = sum(annual_tests.exposure_years)
rate_final = nrow(peaks72) / exposure_final
 $\lambda$  = rate_final

# Use only the Lab05 parameter priors.
Turing.@model function gpd_lab5_prior(y)
    log $\sigma$  ~ Normal(0.0, 3.0)
     $\xi$  ~ Normal(0.0, 0.25)

     $\sigma$  := exp(log $\sigma$ )

    y .~ GPKernel( $\sigma$ ,  $\xi$ )
end

model_baseline = gpd_lab5_prior(excess_final)
n_chains = 4

initial_values = [
    Turing.InitFromParams((
        log $\sigma$ =log(mean(excess_final)),
         $\xi$ =0.1
    ))
    for _ in 1:n_chains
]

chain_baseline = sample(
    MersenneTwister(543),
    model_baseline,
    NUTS(0.99),
    MCMCThreads(),
    2000,
    n_chains;
    initial_params=initial_values,
    progress=false
)

# Summarize diagnostics across all four chains.
function check_chain(chain, label)
    stats = DataFrame(summarystats(chain))
```

```

    worst_rhat = maximum(skipmissing(stats.rhat))
    smallest_bulk_ess = minimum(skipmissing(stats.ess_bulk))
    smallest_tail_ess = minimum(skipmissing(stats.ess_tail))
    divergences = count(!iszero, Array(chain[:numerical_error]))

    @printf(
        "%s: worst  $\hat{R}$  = %.3f, smallest bulk ESS = %.0f, smallest tail ESS = %.0f,
    divergences = %d\n",
        label,
        worst_rhat,
        smallest_bulk_ess,
        smallest_tail_ess,
        divergences
    )

    return stats
end

stats_baseline = check_chain(chain_baseline, "GPD baseline")
display(stats_baseline)

```

```

└ Info: Found initial step size
└    $\epsilon$  = 0.05
└ Info: Found initial step size
└    $\epsilon$  = 0.05
└ Info: Found initial step size
└    $\epsilon$  = 0.05
└ Info: Found initial step size
└    $\epsilon$  = 0.05
GPD baseline: worst  $\hat{R}$  = 1.001, smallest bulk ESS = 2397, smallest tail ESS = 2617,
divergences = 0

```

```

3×10 DataFrame
  Row | param      mean      std      mcse      ess_bulk  ess_tail  rhat  ...
      | VarName... Float64   Float64   Float64   Float64   Float64  Float64 ...
-----|-----
  1 | logσ      3.38563  0.0994635  0.00200185  2467.32  2616.96  1.00041 ...
  2 | ξ          0.169342  0.0774765  0.00158111  2396.73  3015.08  1.00076
  3 | σ          29.6827  2.95525  0.06002    2467.32  2616.96  1.00039
                                     3 columns omitted

```

Parameter estimates and return levels

```
# Use the fit created above instead of the undefined chains_compare.
chain_lab05 = chain_baseline

σ_draws = exp.(vec(Array(chain_lab05[:logσ])))
ξ_draws = vec(Array(chain_lab05[:ξ]))

u = u99

@printf(
    "Retained years: %d\nEvents: %d\nExposure: %.2f years\nλ: %.3f events/year\n",
    length(retained_years),
    nrow(peaks72),
    exposure_final,
    λ
)

parameter_results = DataFrame(
    parameter=["Scale σ (mm)", "Shape ξ"],
    median=[median(σ_draws), median(ξ_draws)],
    lower95=[
        quantile(σ_draws, 0.025),
        quantile(ξ_draws, 0.025)
    ],
    upper95=[
        quantile(σ_draws, 0.975),
        quantile(ξ_draws, 0.975)
    ]
)

# Define return periods by an expected annual exceedance rate of 1/T.
function lab05_return_level(u, σ, ξ, λ, T)
    a = log(λ * T)

    if abs(ξ) < 1e-8
        return u + σ * a
    end

    return u + σ * expm1(ξ * a) / ξ
end

return_results = DataFrame(
    return_period_years=Int[],
    median_mm=Float64[],
    lower95_mm=Float64[],
```

```

        upper95_mm=Float64[]
    )

    for T in (10, 100)
        @assert  $\lambda * T > 1$ 

        rl_draws = lab05_return_level.(u,  $\sigma$ _draws,  $\xi$ _draws,  $\lambda$ , T)

        push!(return_results, (
            T,
            median(rl_draws),
            quantile(rl_draws, 0.025),
            quantile(rl_draws, 0.975)
        ))
    end

    display(parameter_results)
    display(return_results)

    for row in eachrow(return_results)
        @printf(
            "%d-year return level: %.1f mm [%.1f, %.1f], 95%% credible interval\n",
            row.return_period_years,
            row.median_mm,
            row.lower95_mm,
            row.upper95_mm
        )
    end
end

```

```

Retained years: 80
Events: 252
Exposure: 79.98 years
 $\lambda$ : 3.151 events/year
10-year return level: 194.9 mm [172.2, 233.5], 95% credible interval
100-year return level: 340.9 mm [261.8, 519.9], 95% credible interval

```

2×4 DataFrame

Row	parameter	median	lower95	upper95
	String	Float64	Float64	Float64
1	Scale σ (mm)	29.5441	24.232	35.8529
2	Shape ξ	0.165023	0.0273644	0.336081

2x4 DataFrame

Row	return_period_years	median_mm	lower95_mm	upper95_mm
	Int64	Float64	Float64	Float64
1	10	194.91	172.207	233.52
2	100	340.943	261.819	519.893

Repeat the four-chain fit

```
u_fit = u99
y_fit = Float64.(peaks72.prcp_mm) .- u_fit

@assert all(y_fit .> 0)
@assert λ > 0

# Use the same likelihood and priors as the first fit.
Turing.@model function gpd_simple(y)
    logσ ~ Normal(0, 3)
    ξ ~ Normal(0, 0.25)

    σ = exp(logσ)
    y .~ GPKernel(σ, ξ)
end

rng_check = MersenneTwister(20261006)

init_check = [
    Turing.InitFromParams((
        logσ=log(30.0),
        ξ=0.05 + 0.05 * k
    ))
    for k in 1:4
]

chain_check = sample(
    rng_check,
    gpd_simple(y_fit),
    NUTS(0.99),
    MCMCThreads(),
    2000,
    4;
    initial_params=init_check,
    progress=false
)

gpd_diagnostics = check_chain(chain_check, "GPD repeat")
```

```

display(gpd_diagnostics)

σ_check = exp.(vec(Array(chain_check[:logσ])))
ξ_check = vec(Array(chain_check[:ξ]))

rl10_check = lab05_return_level.(
    u_fit, σ_check, ξ_check, λ, 10
)

rl100_check = lab05_return_level.(
    u_fit, σ_check, ξ_check, λ, 100
)

results_check = DataFrame(
    quantity=String[],
    median=Float64[],
    lower95=Float64[],
    upper95=Float64[]
)

for (label, draws) in (
    ("Scale  $\sigma$  (mm)", σ_check),
    ("Shape  $\xi$ ", ξ_check),
    ("10-year return level (mm)", rl10_check),
    ("100-year return level (mm)", rl100_check)
)
    push!(results_check, (
        label,
        median(draws),
        quantile(draws, 0.025),
        quantile(draws, 0.975)
    ))
end

display(results_check)

```

```

└ Info: Found initial step size
└   ε = 0.05
└ Info: Found initial step size
└   ε = 0.05
└ Info: Found initial step size
└   ε = 0.05
└ Info: Found initial step size
└   ε = 0.2

```

GPD repeat: worst $\hat{R} = 1.001$, smallest bulk ESS = 2574, smallest tail ESS = 2742, divergences = 0

2×10 DataFrame

Row	param	mean	std	mcse	ess_bulk	ess_tail	rhat	...
	VarName...	Float64	Float64	Float64	Float64	Float64	Float64	...
1	logσ	3.38153	0.100338	0.00197659	2574.03	2742.05	1.00125	...
2	ξ	0.172045	0.0774306	0.00151598	2630.79	3011.9	1.00091	...

3 columns omitted

4×4 DataFrame

Row	quantity	median	lower95	upper95
	String	Float64	Float64	Float64
1	Scale σ (mm)	29.4531	24.0369	35.7882
2	Shape ξ	0.167632	0.0326143	0.337982
3	10-year return level (mm)	194.798	172.462	233.504
4	100-year return level (mm)	341.943	264.903	521.987

Turning it in

This lab is due on Canvas on Friday, October 9, one week after the Friday session that starts it.

1. Check that your notebook answers each key question and explains each key decision.
2. Render the notebook to a PDF. In the terminal, from the lab folder, run:

```
quarto render index.qmd --to typst
```

This writes `index.pdf` beside the notebook. It runs the whole notebook from the top in a fresh process, so it also confirms your work does not depend on leftover REPL state.

3. Commit and push your work, with GitHub Desktop or by asking Claude to commit and push.
4. Upload `index.pdf` to the Lab 6 assignment on Canvas.
5. Paste the link to your repository in the same Canvas submission.

If the render fails and you cannot fix it, upload what you have and say where it broke. A lab that does not render is worth more to me than a lab you did not turn in.

Note what broke, what Claude got wrong, and what you had to fix by hand, and bring it to the next session.

Bibliography

- [1] D. M. Etheridge, L. P. Steele, R. L. Langenfelds, R. J. Francey, J. Barnola, and V. I. Morgan, “Natural and Anthropogenic Changes in Atmospheric CO₂ over the Last 1000 Years from Air

- in Antarctic Ice and Firn,” *Journal of Geophysical Research: Atmospheres*, vol. 101, no. D2, pp. 4115–4128, Feb. 1996, doi: 10.1029/95jd03410.
- [2] C. MacFarling Meure *et al.*, “Law Dome CO₂, CH₄ and N₂O Ice Core Records Extended to 2000 Years BP,” *Geophysical Research Letters*, vol. 33, no. 14, Jul. 2006, doi: 10.1029/2006gl026152.
- [3] C. Fagnant, A. Gori, A. Sebastian, P. B. Bedient, and K. B. Ensor, “Characterizing Spatiotemporal Trends in Extreme Precipitation in Southeast Texas,” *Natural Hazards*, vol. 104, no. 2, pp. 1597–1621, Nov. 2020, doi: 10.1007/s11069-020-04235-x.
- [4] C. H. Smith, B. Skahill, and D. A. Margo, “Nonstationary Flood Frequency Analysis for Urban Watersheds Using Open-Source Bayesian Software: Contrasting Case Studies from Texas,” *Water*, vol. 18, no. 5, p. 636, Jan. 2026, doi: 10.3390/w18050636.